

46. Permutations Medium

Acceptance: 81.5% | [LeetCode](#)

Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in **any order**.

Example 1:

```
**Input:** nums = [1,2,3]
**Output:** [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Example 2:

```
**Input:** nums = [0,1]
**Output:** [[0,1],[1,0]]
```

Example 3:

```
**Input:** nums = [1]
**Output:** [[1]]
```

Constraints:

- `1 <= nums.length <= 6`
- `-10 <= nums[i] <= 10`
- All the integers of `nums` are **unique**.

Claude Opus Solution

Permutations - Complete Solution

1. Intuition

The key insight is that generating all permutations is a classic **backtracking** problem. We build each permutation **one element at a time**, and at each position we try every unused number.

Think of it as a decision tree: - At each level, we decide which number goes in the current position. - We must track which numbers are already used (since each number appears exactly once per permutation). - When we've placed all numbers, we've completed one permutation — record it and **backtrack** to explore other choices.

The "backtrack" step (undo the choice) is what allows us to reuse the same data structures while exploring all branches.

2. Approach (Pseudocode)

```
permute(nums):
    result = []
    backtrack(current_path = [], used = set())
    return result

backtrack(path, used):
    # Base case: a complete permutation
    if length(path) == length(nums):
        add a COPY of path to result
        return

    # Try every number as the next choice
    for num in nums:
        if num is already used:
            skip
        # Make a choice
        mark num as used
        add num to path
        # Explore deeper with this choice
        backtrack(path, used)
        # Undo the choice (BACKTRACK)
        remove num from path
        unmark num as used
```

3. Python Solution

```
from typing import List
```

```
class Solution:
```

```

def permute(self, nums: List[int]) -> List[List[int]]:
    result = []          # Stores all completed permutations
    used = set()         # Tracks which numbers are currently in our path
    path = []           # The permutation we're currently building

    def backtrack():
        # Base case: path has all numbers -> a full permutation is formed
        if len(path) == len(nums):
            # IMPORTANT: append a COPY (path[:]) because `path` is mutated later.
            # If we appended `path` directly, all entries would point to the
            # same list and end up empty after backtracking.
            result.append(path[:])
            return

        # Decision: try each number as the next element in the permutation
        for num in nums:
            if num in used:
                continue          # Skip numbers already placed

            # --- Make the choice ---
            used.add(num)         # Mark as used
            path.append(num)      # Add to current permutation

            # --- Explore further with this choice ---
            backtrack()

            # --- Undo the choice (backtrack) ---
            path.pop()           # Remove from current permutation
            used.remove(num)      # Mark as unused again

    backtrack()
    return result

```

Alternative: Index-Swapping (no extra used set)

```

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        result = []

        def backtrack(start):
            # When start reaches end, nums is fully arranged
            if start == len(nums):
                result.append(nums[:])    # copy current arrangement
                return

            for i in range(start, len(nums)):
                # Fix nums[i] at position `start` by swapping
                nums[start], nums[i] = nums[i], nums[start]
                backtrack(start + 1)      # permute the rest
                # Swap back to restore original order (backtrack)
                nums[start], nums[i] = nums[i], nums[start]

            backtrack(start + 1)

        backtrack(0)
        return result

```

4. Complexity Analysis

Let $n = \text{len}(\text{nums})$.

Time Complexity: $O(n \times n!)$

- There are $n!$ permutations to generate.
- For each permutation, we spend $O(n)$ time copying it into the result (`path[:]`).
- The total work in the recursion tree is bounded by the number of leaves $\times$ cost per leaf, giving $O(n \times n!)$.

Space Complexity: $O(n)$ (excluding output)

- The recursion depth is at most n .
- The `path` list and `used` set each hold at most n elements.
- Including the output, space is $O(n \times n!)$ since we store all permutations.

5. Alternative Approaches

Approach	Idea	Notes
Backtracking + used set (main)	Track chosen elements explicitly	Clean, intuitive, works with any value range
In-place swapping	Swap elements to fix positions	No extra <code>used</code> set; mutates input temporarily
Iterative insertion	Insert each new number into every position of existing partial permutations	Build-up approach; e.g., <code>[1]</code> → <code>[2,1]</code> , <code>[1,2]</code> → ...
<code>itertools.permutations</code>	Built-in library	<code>return [list(p) for p in itertools.permutations(nums)]</code> — concise but doesn't demonstrate the algorithm

Iterative Insertion Example

```
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        perms = [[]]
        for num in nums:
            new_perms = []
            for p in perms:
                # Insert `num` at every possible position in p
                for i in range(len(p) + 1):
                    new_perms.append(p[:i] + [num] + p[i:])
            perms = new_perms
        return perms
```

Key Takeaways

- **Backtracking pattern:** choose → explore → un-choose. This pattern solves combinations, subsets, N-Queens, and many others.
- Always **copy** the result (`path[:]`) when storing mutable state.
- The swapping approach saves the auxiliary `used` set but only works because we permute the array in place.

Codex Solution

1. Intuition

We need to generate every possible ordering of the given distinct numbers.

This is a classic **backtracking** problem.

At each position in the permutation, we choose one number that has not been used yet. Then we recursively fill the next position. Once the current permutation has length `n`, we have formed one complete valid permutation.

The key idea:

Build permutations step by step, and undo choices after exploring them so we can try other possibilities.

For example, for `nums = [1, 2, 3]`:

```
Start: []
Choose 1 -> [1]
  Choose 2 -> [1, 2]
    Choose 3 -> [1, 2, 3]
  Backtrack
  Choose 3 -> [1, 3]
    Choose 2 -> [1, 3, 2]
Choose 2 -> [2]
...
```

This "choose → explore → unchoose" pattern is the core of backtracking.

2. Approach

We maintain:

- `path`: the current partial permutation
- `used`: a boolean array where `used[i]` tells whether `nums[i]` is already in `path`
- `result`: stores all complete permutations

Pseudocode

```
function permute(nums):
    result = []
    path = []
    used = array of false values with length nums.length

    function backtrack():
        if path length == nums length:
            add a copy of path to result
            return

        for i from 0 to nums.length - 1:
            if used[i] is true:
                continue

            mark nums[i] as used
            add nums[i] to path

            backtrack()

            remove nums[i] from path
            mark nums[i] as unused

    backtrack()
    return result
```

3. Python Solution

```
from typing import List

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        # Stores all complete permutations
        result = []

        # Current permutation being built
        path = []

        # used[i] tells us whether nums[i] is already in path
        used = [False] * len(nums)

        def backtrack() -> None:
            # Base case:
            # If the current path contains all numbers,
            # we have formed one complete permutation.
            if len(path) == len(nums):
                # Append a copy of path because path will be modified later
                result.append(path[:])
                return

            # Try placing each unused number at the current position
            for i in range(len(nums)):
                # Skip numbers already used in the current permutation
                if used[i]:
                    continue

                # Choose nums[i]
                used[i] = True
                path.append(nums[i])
```

```

        # Explore further choices
        backtrack()

        # Undo the choice: backtrack
        # This allows nums[i] to be used in a different position later
        path.pop()
        used[i] = False

    backtrack()
    return result

```

4. Complexity Analysis

Let `n = len(nums)`.

Time Complexity: $O(n * n!)$

There are $n!$ permutations.

For each complete permutation, we copy the current path into the result:

```
result.append(path[:])
```

Copying takes $O(n)$ time.

Therefore:

```
Total time = O(n * n!)
```

Space Complexity: $O(n * n!)$

The output itself contains $n!$ permutations, each of length n , so storing the result takes:

```
O(n * n!)
```

Auxiliary space excluding the output is:

```
O(n)
```

because:

- `path` can have at most n elements
- `used` has length n
- recursion depth is at most n

So:

```
Auxiliary space = O(n)
Total space including output = O(n * n!)
```

5. Alternative Approaches

1. In-place Swapping Backtracking

Instead of using a `used` array, we can generate permutations by swapping elements in the original array.

Idea:

```

Fix one position at a time.
Swap every possible number into that position.
Recurse on the next position.
Swap back afterward.

```

This avoids the extra `used` array, but still uses recursion.

```

from typing import List

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        result = []

        def backtrack(start: int) -> None:

```

```

# If start reaches the end, nums is one valid permutation
if start == len(nums):
    result.append(nums[:])
    return

for i in range(start, len(nums)):
    # Put nums[i] at index start
    nums[start], nums[i] = nums[i], nums[start]

    # Recurse to fill the next index
    backtrack(start + 1)

    # Swap back to restore original order
    nums[start], nums[i] = nums[i], nums[start]

backtrack(0)
return result

```

Complexity is the same:

```

Time:  $O(n * n!)$ 
Space:  $O(n)$  auxiliary,  $O(n * n!)$  including output

```

2. Using Python's `itertools.permutations`

Python has a built-in function for permutations:

```

from typing import List
from itertools import permutations

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        return [list(p) for p in permutations(nums)]

```

This is concise, but for interviews and algorithm practice, the backtracking solution is usually expected.